

# eTCP – Part 1: Connection Management (Connection Setup and Teardown)

## 1. Overview

eTCP (Educational TCP) is a user-level TCP stack for education. Unlike standard TCP in the Linux kernel, all state management, packet handling, and control logic run in user space. In part 1, you will implement the logic for **TCP Connection Setup (3-way handshake)** and **Teardown (4-way handshake)** for the eTCP stack. To do this assignment, you need to implement state-based handlers called from *ProcessTCPPacket*.

Before starting part 1, we highly encourage you to read **eTCP\_appendix**, which explains the eTCP architecture, its core data structures, and the set of APIs you must employ for this assignment.

## 2. Connection Setup Overview

TCP uses a three-way handshake to synchronize sequence numbers and confirm endpoint readiness. The three-way handshake consists of the following steps:

### Step 1 — SYN

The client initiates the connection by sending a SYN packet.

- The SYN packet contains the client's Initial Sequence Number (ISN).
- The client enters the SYN\_SENT state.

### Step 2 — SYN + ACK

When the server receives the SYN packet:

- The server allocates a TCP control block (connection state) and records the client's sequence number as Initial Receive Sequence (IRS).
- The server sends back a SYN+ACK packet that contains the server's own ISN and an acknowledgement of the client's SYN.
- The server enters the SYN\_RCVD state

### Step 3 — ACK

The client receives the SYN+ACK and sends the final ACK.

This packet:

- Acknowledges the server's SYN and completes the handshake

At this point:

- The client enters the ESTABLISHED state, and when the server receives the ACK, it enters the ESTABLISHED state
- The TCP connection is ready to exchange data.

### 3. Connection Setup in eTCP

This section explains how eTCP implements connection setup at the code level. An eTCP application employs two types of threads: the **Application thread** that implements the application logic using the eTCP socket API, and the **eTCP thread** that implements the core eTCP stack itself. The application thread issues its own socket API calls such as: *etcp\_listen*, *etcp\_accept*, *etcp\_connect*, while the eTCP thread processes packets and implements the TCP state machine. The two threads communicate through shared structures such as *tcp\_flow* objects, event queues, and send/control lists. More information is found in eTCP\_appendix.

#### Passive Open in eTCP (Server Side)

Passive open means that the server is waiting for incoming connections. The following steps describe how passive open is implemented in eTCP.

Step 1 — The Application thread prepares the listening socket

The server application executes: *etcp\_socket* → *etcp\_bind* → *etcp\_listen*. Now the application thread blocks inside *etcp\_epoll\_wait*. It waits until a new connection becomes available.

Step 2 — SYN arrives at the eTCP thread

The eTCP thread receives and processes a SYN packet in the TCP input path (*ProcessTCPPacket*). Since no flow is registered yet, *CreateFlow* is invoked to allocate and initialize a new *tcp\_flow* structure for the connection. It initializes the receive and send variables and sets the connection state to TCP\_ST\_LISTEN. Then, *LISTEN\_Handler* is called.

Step 3 — SYN+ACK is scheduled

The eTCP thread on the server must respond with a SYN+ACK packet. Instead of sending it immediately, eTCP schedules the packet by inserting the flow into the control list using *RegisterFlowToControlPktList*. Then, during the send phase of the main loop, *FlushControlPktList* is invoked, which calls *MakeControlPacket* that generates the SYN+ACK packet (*MakeTCPPacket*). Finally, the eTCP thread transmits it (*send\_pkts*).

Step 4 — Final ACK arrives

When the eTCP thread receives an ACK packet from the client, the connection state changes to TCP\_ST\_ESTABLISHED.

Step 5 — Notify the application

After the connection is established, the eTCP stack should notify the Application thread. This is done by generating a read event on the listening socket using *AddEventToEpollQ*. The event is inserted into the event queue and delivered to the Application thread. The blocked *etcp\_epoll\_wait* call wakes up, and *etcp\_accept* can be invoked to retrieve the newly established connection from the accept queue.

### Active Open in eTCP (Client Side)

Active open corresponds to the client initiating the connection.

Step 1 — The Application thread on the client side calls *etcp\_connect*  
Inside this function:

1. A new *tcp\_flow* is created(*CreateFlow*)
2. The state is set to *TCP\_ST\_SYN\_SENT*
3. Newly created *tcp\_flow* is enqueued to *connectq*(*EnqueueToFlowQ*)

Step 2 — SYN is transmitted

In the main loop of the eTCP thread, *ProcessAppRequests* dequeues a *tcp\_flow* from *connectq* (*DequeueFromFlowQ*) and adds it to the control list (*RegisterFlowToControlPktList*). Then, during the send phase of the eTCP main loop, *FlushControlPktList* is invoked, which prepares a SYN packet. Finally, the eTCP thread sends it via *send\_pkts*.

Step 3 — SYN+ACK arrives

When the client's eTCP thread receives a SYN+ACK packet, it calls *SYN\_SENT\_Handler* to process it in the TCP input path.

Step 4 — Connection ready

The Application thread waiting in *etcp\_epoll\_wait* is notified through an EPOLLOUT event, indicating that the socket is ready for writing.

Step 5 — Final ACK is sent

During the send phase of the eTCP thread, an ACK packet is made and transmitted.

## 4. Connection Teardown Overview

After a TCP connection has finished exchanging data, the connection must be gracefully terminated. TCP uses a four-way handshake to close a connection because TCP connections are full-duplex, meaning each direction of the connection must be closed independently.

The teardown process ensures that:

- All outstanding data has been delivered
- Both sides agree to close the connection

The TCP connection termination procedure consists of four steps.

#### Step 1 — FIN

The side that wants to close the connection(let's call this **A**) sends a FIN packet. This indicates that **A** has no more data to transmit. Then, **A** enters the state of FIN\_WAIT\_1.

#### Step 2 — ACK

The receiver (let's call it **B**) acknowledges the FIN with an ACK. **B** enters the CLOSE\_WAIT state. **A** receives this ACK and transitions to the FIN\_WAIT\_2 state.

At this point:

- **A** has closed its sending direction
- **B** can still send data

#### Step 3 — FIN

When **B** is ready to close the connection, it sends its own FIN. **B** enters the LAST\_ACK state.

#### Step 4 — Final ACK

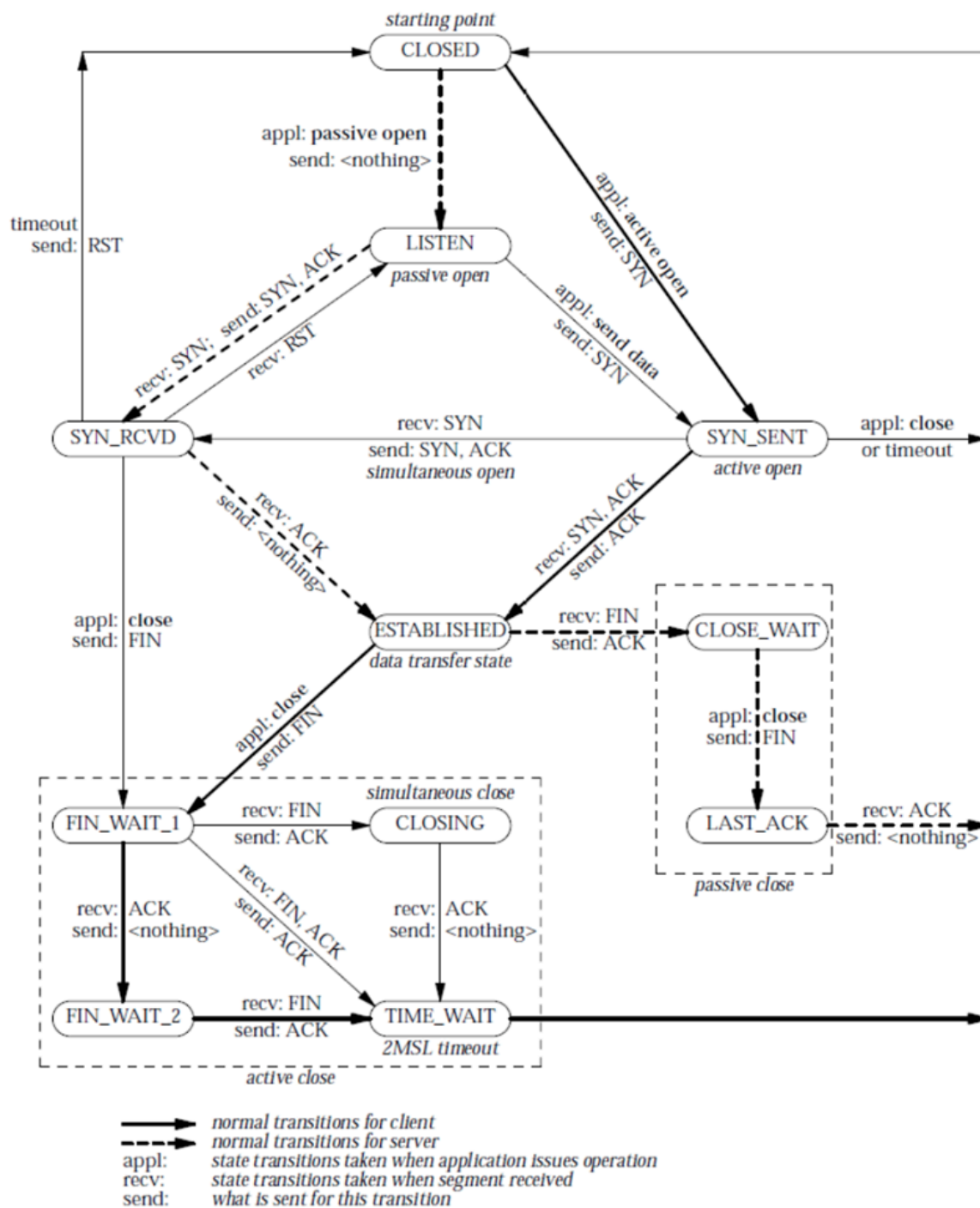
**A** acknowledges **B**'s FIN. Then, **A** enters the TIME\_WAIT state. If **B** successfully receives an ACK for its FIN, it enters the CLOSED state. As you saw, after sending the final ACK, **A** remains in the TIME\_WAIT state for a certain duration.

This state exists to ensure that:

1. **A** retransmits the final ACK if it receives the retransmitted FIN from **B** (final ACK loss)
2. Any delayed packets from the old connection are discarded.

If the final ACK is lost, the **A** may retransmit its FIN, and the **B** must still be able to respond. After the TIME\_WAIT timer expires, the connection is fully removed.

Please refer to the figure below.



## 5. Connection Teardown in eTCP

This section explains how the eTCP stack implements connection teardown. eTCP follows the standard TCP state machine. Like in the regular TCP application, the Application thread triggers closure using *etcp\_close*. The eTCP thread handles FIN transmission, state transitions, timeout handling, and retransmissions as needed.

## Active Closure in eTCP

Active closure starts when the application explicitly closes a socket.

Step 1 — Application calls *etcp\_close*

Inside this function, the corresponding *tcp\_flow* is enqueued to *closeq* (*EnqueueToFlowQ*). Then, inside *CoreLoop*, *ProcessAppRequests* is invoked. Inside this function, the flow is dequeued from *closeq* and added to the *ControlList* (*RegisterFlowToControlPktList*).

Step 2 — FIN transmission

During the send phase of the main loop, *FlushControlPktList* invokes *MakeControlPacket*, and it constructs the FIN packet using *MakeTCPPacket*. Finally, the FIN packet is transmitted via *send\_pkts*. A retransmission timer may also be registered to handle FIN loss.

Step 3 — ACK for FIN arrives

When the ACK for the FIN arrives, the eTCP thread processes the packet and transitions into the state of *FIN\_WAIT\_2*.

At this stage:

- The local side has finished sending data
- The remote side may still send remaining data

Step 4 — Remote FIN arrives

When the remote endpoint finishes sending data, it sends a FIN. The eTCP thread then processes this packet via *FIN\_WAIT\_2\_Handler*.

1. Updates the receive sequence number
2. Adds flow to Control List to schedule sending ACK for this  
*FIN(RegisterflowToControlPktList)*
3. Adds flow to TimeWait List(*RegisterToTimewaitList*)
4. Transition the connection state to *TIME\_WAIT*
5. Send ACK of FIN to remote peer

## Passive Closure in eTCP

Passive closure begins when the remote peer initiates connection termination.

Step 1 — FIN received

When the eTCP thread receives a FIN packet, followings happen:

- *rcv\_nxt* is updated
- The eTCP thread adds the flow to the control list to schedule sending  
*ACK(RegisterflowToControlPktList)*
- It raises a read event to notify the application(*RegisterReadEvent*)
- State transitions into the *CLOS\_WAIT* state

Now in the main loop(*CoreLoop*), *FlushEventQueue* function moves read event(generated via *RegisterReadEvent*) to the user event queue and wakes up the Application thread waiting in *etcp\_epoll\_wait*. Inside *etcp\_read*, when the current flow is in the CLOSE\_WAIT state:

- If the receive buffer is empty, *etcp\_read* returns 0, indicating EOF. The application interprets this as connection closure and calls *etcp\_close*.
- If the receive buffer still contains data, *etcp\_read* first delivers the remaining data to the application. After the last data is consumed, *etcp\_read* re-arms an EPOLLIN event(via *AddEventToEpollQ*), so that the application will be notified again and perform another read. On that next read, *etcp\_read* returns 0 (EOF).

#### Step 2 — Application closes the socket

Inside *etcp\_close*, the flow is enqueued to *closeq*, and later it is processed in *ProcessAppRequests* in the main loop (*CoreLoop*) of the eTCP thread. TCP state transitions into the LAST\_ACK state, and the flow is inserted into the control list (*RegisterflowToControlPktList*).

#### Step 3 — FIN|ACK packet is sent

During the send phase, *FlushControlPktList* is invoked, and FIN+ACK is made (*MakeTCPPacket*). Via *send\_pkts*, it is sent to the peer.

#### Step 3 — Final ACK received

When the remote peer acknowledges the FIN, *LAST\_ACK\_Handler* is invoked. The connection transitions into the CLOSED state, and the TCP flow structure is destroyed.

**\*\* You may find a detailed explanation to handle TIME\_WAIT in eTCP\_Appendix-timer management in the eTCP.**

## 6. What should I do?

Now, you are ready to fill in the TCP logic. Source code files you need to update are:

- tcp\_in.c
- tcp\_out.c
- timer.c

All student-editable sections are wrapped in */\*\*\*\*\* \*/* in the code. Below is a description of what each section must accomplish.

Let's first take a look at TODO in tcp\_in.c.

### 1. Passive Open: How the Server Accepts a New Connection

Assume the server application has already called *etcp\_socket()*, *etcp\_bind()*, and *etcp\_listen()*. At this point, the Application thread is simply waiting for an incoming connection, typically through *etcp\_epoll\_wait()* and eventually calls *etcp\_accept()*.

Now consider what happens when a client sends a SYN packet.

The packet first enters the eTCP receive path through *ProcessTCPPacket*. Since this is the first packet of the connection, there is no existing flow in the flow table. As a result, *CreateHashTableEntry* is called. For a valid incoming SYN, this function invokes *OpenPassive* to create and initialize a new *tcp\_flow*.

This is the first point where you need to complete the skeleton. In *OpenPassive*, you should initialize the flow using the information in the incoming SYN. In particular, you should:

- Record the peer's initial sequence number as *irs*
- Initialize the expected receive sequence number
- Store the peer's advertised window
- Initialize any minimal sender-side state required by later control-path logic

Once the flow has been created, the connection enters the normal TCP state-dispatch path. The initial state is *TCP\_ST\_LISTEN*, so the packet is handled by *LISTEN\_Handler*.

This function represents the server's first protocol response. When a valid SYN is received, the server should:

- Advance the receive-side state appropriately
- Transition from *TCP\_ST\_LISTEN* to *TCP\_ST\_SYN\_RCVD*
- Schedule transmission of a SYN+ACK by inserting the flow into the *ControlList*

After the SYN+ACK has been sent, the server remains in *TCP\_ST\_SYN\_RCVD* while waiting for the final ACK from the client. When that ACK arrives, *ProcessTCPPacket* dispatches it to *SYN\_RCVD\_Handler*.

This function completes the passive-open path. Here, your implementation should:

- Verify that the ACK correctly acknowledges the previously sent SYN+ACK
- Update the sender-side acknowledgment variables (*snd\_una*, *snd\_nxt*)
- Reset retransmission-related state if needed
- Remove the retransmission timer associated with the earlier control packet
- Transition the flow state to the *TCP\_ST\_ESTABLISHED* state

Once the connection reaches *TCP\_ST\_ESTABLISHED*, the rest of the passive-open path is already supported by the provided code. The flow is inserted into the listener's accept queue, and a read event is generated for the listening socket. This wakes the application thread, allowing *etcp\_accept* to return the newly established connection.

That is the complete server-side connection setup path you are expected to implement.

## 2. Active Open: How the Client Initiates a Connection

The client side begins differently. Instead of waiting for an incoming packet, the connection begins when the application explicitly requests one by calling *etcp\_connect*.

This is the active open case.

At a high level, *etcp\_connect* creates a flow, places it in `TCP_ST_SYN_SENT`, and schedules transmission of a SYN packet through the control path. The actual packet transmission is deferred to the send phase, just as in the server case.

From the receive path perspective, the key moment occurs when the server replies with a SYN+ACK.

That reply is processed in *SYN\_SENT\_Handler*, which is the main handler for active open. This function must handle several different cases, but the most important one is the successful handshake case, where the packet contains both SYN and ACK.

Before handling success, the function first validates the ACK field. If the ACK is not acceptable, the packet should be ignored or answered with an RST, depending on the situation. If the packet carries RST, the connection attempt should fail.

If the packet contains a valid SYN+ACK, the connection should be completed through *OpenActive*. This helper is responsible for initializing the flow using the information learned from the peer. At this point, the client knows the server's initial sequence number, so the relevant receive-side variables can be initialized. In particular, this includes values such as:

- `irs`
- `rcv_nxt`
- `snd_nxt`
- `peer_wnd`

After *OpenActive* returns successfully, the rest of the state transition is performed in *SYN\_SENT\_Handler*. The implementation should:

- reset retransmission counters
- remove the retransmission timer for the previously sent SYN
- transition the flow state to `TCP_ST_ESTABLISHED`
- raise a write event to notify the application that the connection is now ready
- schedule the final ACK by inserting the flow into the control list

The write event is important. In eTCP, as in the standard non-blocking socket semantics, a successful *connect* is exposed to the application as write readiness. That is why *RegisterWriteEvent* is used here.

That completes the client-side connection setup path.

### 3. ACK Handling in Connection Management

Whenever a packet carrying an ACK is received, *HandleACK* is invoked. For Part 1, *HandleACK* is intentionally simplified. You are only expected to handle ACK processing for control packets, not for data transfer. That means your concern here is limited to acknowledgments of SYN, SYN+ACK, and later FIN.

### 4. What a Correct Implementation Should Achieve

If your setup logic is implemented correctly, the following behaviors should hold.

On the server side:

- An incoming SYN creates a new flow
- The flow transitions through LISTEN and SYN\_RCVD
- The server sends SYN+ACK
- The final ACK transitions the flow to ESTABLISHED
- The listening application is notified and can accept the connection

On the client side:

- *connect* causes a SYN to be sent
- A valid SYN+ACK completes the handshake
- The flow transitions to ESTABLISHED
- The application is notified that the socket is writable

If these two paths work correctly, then the three-way handshake portion of Part 1 is complete.

## 5. Connection Teardown

The teardown path should be handled in the same way, by following the state machine one packet at a time. You will apply the same reasoning to questions such as:

- What should happen when a FIN is received in ESTABLISHED?
- How should FIN\_WAIT\_1 react to an ACK of its FIN?
- When should the connection move to CLOSE\_WAIT, LAST\_ACK, FIN\_WAIT\_2, or TIME\_WAIT?

For this assignment, you don't need to care about "simultaneous close" that transitions into the CLOSING state. Due to the complexity, we handle this case in the skeleton code.

## 6. TCP output path

Now, let's take a look at `tcp_out.c`.

The other half of connection management is the transmit path, which determines how control packets (SYN, ACK, FIN, RST) are generated and sent. In this project, the transmit-side logic for control packets is mainly handled by two functions in `tcp_out.c`:

- *MakeTCPPacket*
- *MakeControlPacket*

These functions serve different purposes.

- *MakeTCPPacket* is responsible for constructing a TCP segment and filling in the header fields.
- *MakeControlPacket* determines which control packet to generate based on the flow's current TCP state.

Understanding the interaction between these two functions will make it much easier to follow the connection setup and teardown logic.

*MakeTCPPacket* builds an outgoing TCP segment using the information stored in the current *tcp\_flow*. The function first allocates space for the packet through *MakeIPPacket*, then initializes the TCP header fields.

Some fields are already handled for you, including:

- source and destination ports
- base sequence number selection
- TCP option length calculation
- checksum generation

Your implementation mainly needs to complete the handling of the control flags. In particular, you should consider how the following flags affect the TCP header and flow state:

- SYN
- ACK
- RST

When processing the ACK flag, remember that the acknowledgment number should reflect the next sequence number expected from the peer. The flow also maintains timestamps related to ACK transmission and activity tracking. Another important detail concerns sequence number space. In TCP, certain control flags consume sequence numbers:

- SYN consumes one byte
- FIN consumes one byte

Additionally, when a segment containing SYN or FIN is sent, the retransmission timer for that control packet must be scheduled.

While *MakeTCPPacket* constructs the actual segment, *MakeControlPacket* determines which type of packet to generate. This function examines the current TCP state and calls *MakeTCPPacket* with the appropriate flags.

You will see several TCP states handled in this function. Your task is to complete the mapping between TCP states and the control packets that should be sent in each state. The receive-side logic already places flows into the control list when a control response is required. Later, the send path processes that list and invokes *MakeControlPacket* to construct the corresponding segment.

## 7. Retransmission Handling

Finally, you will need to implement simple retransmission logic in *timer.c*. Specifically, you will implement the *ProcessRetransmissionTimeout* function. In this part, retransmission handling should remain limited to connection-management packets only. That means you only need to consider retransmission of outstanding control packets, rather than payload data. You may find the instructions in the skeleton code.

## 7. How to Test My Code?

For your two designated machines, you should normally work only on machine 1, which is either nw01 or nw03. The other machine(machine 2), nw02 or nw04, is normally used only to run applications on the standard Linux TCP stack. However, for debugging purposes, you can run eTCP applications on machine 2. I will explain it in a later section.

First, go to the **ETCP/** directory and open **set\_up.sh**. Enter your machine number in that file, then run the script. This will automatically generate the appropriate ARP configuration file in the etcp/example directory. You do not need to understand ARP for this assignment.

```
./set_up.sh
```

After you finish implementing your TCP stack and want to test it, go to the **etcp/src** directory and run **make**. Then move to the **example/part1** directory and run **make** again. This will build the ETCP\_server and ETCP\_client applications.

### [1] eTCP Client - Linux Server

Since you have two machines available for testing, run the eTCP client on one machine and the Linux server on the other.

**\*\* If you are using nw01 and nw02, run eTCP application on nw01!  
if you are using nw03 and nw04, run eTCP application on nw03!**

Assume you are in the example/part1 directory. To run the eTCP client, first open *ETCP.conf* and modify *the ip* field to match your IP address (the designated IP address for the machine you are trying to run the eTCP application on).

Next, open the **run\_ETCP\_client.sh** script.

For argv[1], enter the IP address of the other machine running the remote server.

For argv[2], enter your designated port number, which is the port number of the remote server you want to connect to.

For argv[3], enter the number of concurrent flows to create. For example, if you enter 5, the client will establish 5 connections and immediately close them. The number of concurrent flows must not exceed 5.

**\*\*Do not modify any arguments other than the three described above.\*\***

After completing these steps, open a terminal and execute:

```
./run_ETCP_client.sh
```

Make sure that the server (linux\_server) is already running on the other machine before starting the eTCP client. To run a Linux server on the other machine, go to **linux\_app/part1**. Then, compile the code.

```
$ gcc linux_server.c -o linux_server
```

Find **run\_linux\_server.sh** and set the application's arguments. Then run it on the terminal.

```
./run_linux_server.sh
```

## [2] eTCP Server - Linux Client

It is similar to case [1]. After successfully building binaries, find ***run\_ETCP\_server.sh***. Set appropriate arguments for it and run it in the terminal.

```
$/run_ETCP_server.sh
```

On the other machine, compile ***linux\_client.c*** and set appropriate arguments in ***run\_linux\_client.sh***. Then run it in the terminal.

```
$/run_linux_client.sh
```

## [3] eTCP - eTCP Testing for Debugging

In addition to the two standard test cases above, you may also run eTCP on both machines for debugging purposes.

This mode is intended only as an additional debugging tool. The official/basic test setup for this assignment is still:

- machine 1: eTCP application
- machine 2: Linux TCP application

However, when you want to isolate bugs inside your own implementation, it can be useful to run an eTCP application on both machines and let the two eTCP endpoints communicate directly.

In the spreadsheet, an additional **Debugging Port** is provided for each student. If you want to test communication between two eTCP endpoints, use the debugging port assigned to your pair in the spreadsheet. This port is reserved for eTCP-eTCP testing. When using this mode, make sure that:

- ETCP.conf is configured correctly on both machines,
- both sides use the designated IP addresses of the two machines,
- both applications use the eTCP-to-eTCP debugging port assigned in the spreadsheet.
- run ***./set\_up.sh*** with each machine number

This mode can be useful for debugging handshake, teardown, retransmission, ACK processing, and TCP state transitions between two eTCP endpoints.

Note that successful eTCP-to-eTCP communication does not replace the required tests against the Linux TCP stack. You should still verify that your implementation works correctly in cases [1] and [2]. Also, eTCP-to-eTCP communication will not be evaluated.

## [4] Simulating Harsh Conditions

In addition to the normal test scenarios above, you are encouraged to test your implementation under harsher network conditions. In real networks, packets may be lost, and connections may require retransmissions. A TCP implementation that appears correct in

ideal cases may still fail when packet loss occurs during connection establishment, termination, or data transfer. These options have the following meanings:

- `drop_syn_nth`  
Drops the outgoing SYN packet *n* times. This is useful for testing whether connection establishment can recover from a lost SYN. Useful when you are testing **eTCP\_client**.
- `drop_synack_nth`  
Drops the outgoing SYN-ACK packet *n* times. This is useful for testing whether connection establishment still succeeds when the server's reply is lost. Useful when you are testing **eTCP\_server**.
- `drop_fin_nth`  
Drops the outgoing FIN packet *n* times. This is useful for testing whether connection teardown can recover correctly when a FIN is lost. Useful when you are testing **eTCP\_client**.

A value of 0 disables the corresponding fault injection option. For example:

- `drop_syn_nth=3` means that the outgoing SYN packet will be dropped 3 times.
- `drop_fin_nth = 1` means that the outgoing FIN packet will be dropped 1 time.

For Part 1, these options can be used to check whether your implementation correctly handles retransmissions during connection establishment and teardown. Set these values from 0 to 3 and test the robustness of your implementation.